This code does two powerful things for us. First, the `form_with` helper on the first line sets up an HTML form that knows about Rails routes and models. The argument, `model: order`, tells the helper which instance variable to use when naming fields and sending the form data back to the controller.

The second powerful feature of the code is how it creates the form fields themselves. You can see that `form_with` sets up a Ruby block environment (that ends on the last line of the listing with the `end` keyword). Within this block, you can put normal template stuff (such as the `<p>` tag). But you can also use the block's parameter (`form` in this case) to reference a form context. We use this context to add a text field with a label by calling `text_field` and `label`, respectively. Because the text field is constructed in the context of `form_with`, it's automatically associated with the data in the `order` object. This association means that submitting the form will set the right names and values in the data available to the controller, but it will also pre-populate the form fields with any values already existing on the model.

All these relationships can be confusing. It's important to remember that Rails needs to know both the *names* and the *values* to use for the fields associated with a model. The combination of `form_with` and the various field-level helpers (such as `text_field`) gives it this information.

Now we can update the template for the form that captures a customer's details for checkout. It's invoked from the `new` action in the order controller, so the template is called `new.html.erb`, found in the `app/views/orders` directory:

rails80/depot_o/app/views/orders/new.html.erb

```
<% content_for :title, "New order" %>

<div class="md:w-2/3 w-full">
  <h1 class="font-bold text-4xl">Please Enter Your Details</h1>

  <%= render "form", order: @order %>
</div>
```